For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

Approaching a Design problem

DSA Interviews

- Problem statement is concrete - well defined
- Example input-output pairs
- Idea about the kind of solution that will work
    - input constraints
    - input: array of integers of size 10^6
    - can you use a $O(n^2)$ algo? No. Max $O(n \log n)$

1. Problem Statement
2. Examples - solve it by hand
3. Brute Force
4. **Draw observations**
5. Optimized

Design Interviews

- Problem statement is extremely vague
  "Design Typeahead"
- you might not even know what typeahead is
- interviews are very open ended
    - discussion can go in any direction
    - depends on your expertise
    - depends on the interviewer's expertise
- interviews are still time constrained
    - 45 mins
- you must
    - what you're supposed to build
    - figure out what the interviewer wants
    - build that solution
    - take sufficient complex things so that you can showcase your skills & impress the interviewer

The 5-step design approach

(20-25 mins)

1. Problem Statement
2. Functional Requirements
3. Non-functional Requirements
4. Scale Estimation

(25-20 mins)

5. **System Design**

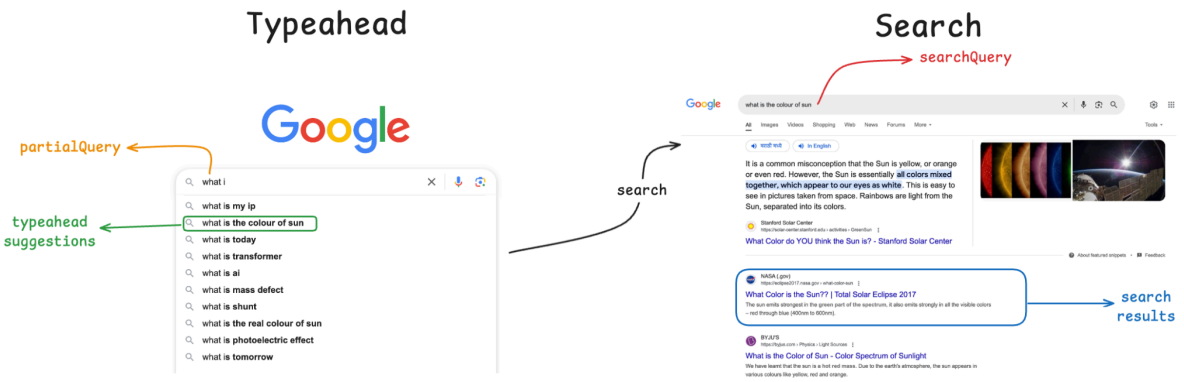
Problem Statement

Reason by analogy

Find existing companies / systems that offer a similar product / feature.
Gives you an overview of the umbrella of different scopes that you can consider.

Typeahead is a specific form of autocomplete.
Try to type-ahead of the user

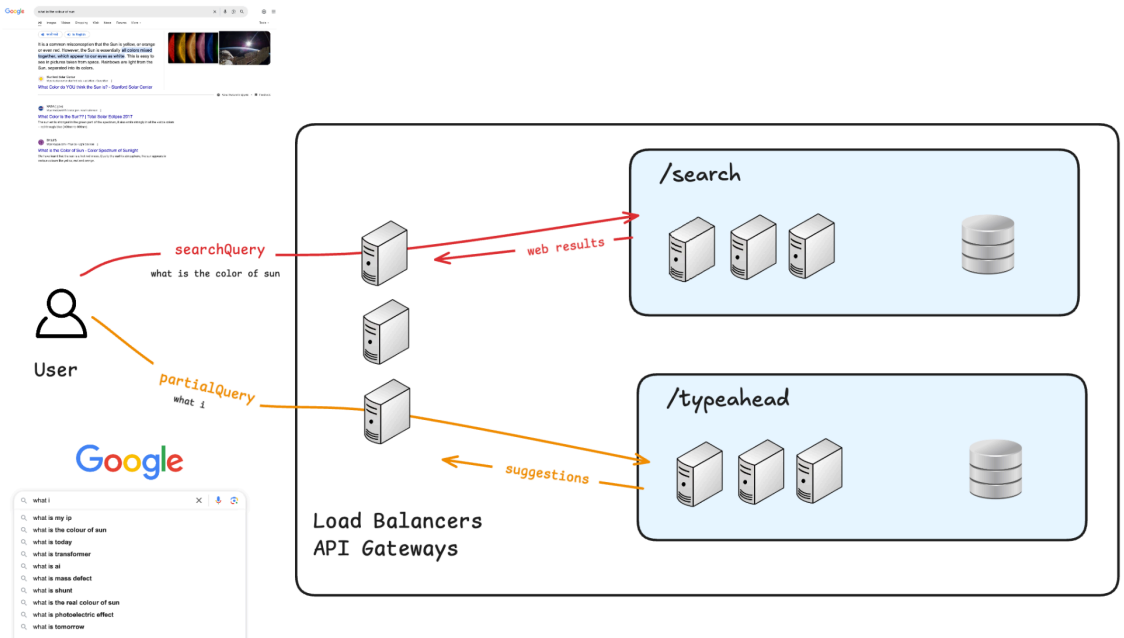
Typeahead in Google



Typeahead in Amazon

Typeahead & Search are different products

but typeahead is always followed by with some form of search.



"If you can't dazzle them with your brilliance, baffle them with your bullshit!"

What not to build

Autocomplete is available in many different settings

1. **Smartphone Keyboards:** word completion / contacts / ..
2. **Text editors / Code editors:**
    a. intellisense: complete next token
    b. Github CoPilot: can show entire functions
3. **Grammarly:** sentence completion while prose writing
4. **Gmail / Outlook:**
    a. sentence completion for writing mail
    b. to/cc/bcc: email completion from contacts

Key difference b/w Typeahead & Autocomplete

Autocomplete is typically local to your machine. Any suggestions are specific to you. Autocomplete is standalone.

Typeahead is always associated with search.
The list of suggestions in typeahead are the most popular queries searched globally in the past, that match the given prefix.

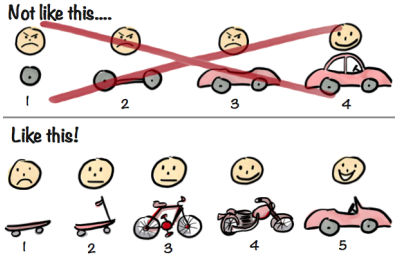
Functional Requirements (FR)

Functional requirements are about the "functionality" that is being offered to the clients/actors

Do NOT go on a feature frenzy — your task is NOT to suggest all the features in the world. Limit yourself to the MVP features — so that we can design them within our limited time

Minimal Viable Product (MVP)

1. **Minimal:** minimal set of features to achieve viability
2. **Viable:** should demonstrate the core features & usability
3. **Product:** solve a genuine problem



Actors

1. **End User:** the person doing the google search

MVP Features

Always keep the "minimal" in your mind. MVP features is not a feature suggestion competition.

MVP features perfect for discussion (v0)	**Future Scope** good to have, but not absolutely necessary (v1+)	**Bad** get yourself rejected in an interview (v-never)
As the user is typing in the search-box, we should auto-complete their search query. *We should show "typeahead" suggestions.* `typeahead(partial_query) => suggestions_list` user-facing	Typeahead results should be personalized 1. geolocation 2. user's browsing/search history	Display a textbox in which the user can type something (frontend)
The suggestions shown in typeahead are popular searches that other people have made in the past `log_search(search_query) => void` internal	Results should take recency/news factor into consideration *(trending topics)*	Clicking a typeahead suggestion should lead to a search (frontend)
We will limit to showing only ***top 5***, ***relevant*** suggestions. • **relevant:** any suggestion should be a *prefix match* with the partial_query • **top 5:** we will keep track of the number of times a `search_query` has been searched for (globally) — basically keep track of the "popularity" of a search query. Given a `partial_query`, we will show the search_queries that are prefix match with `partial_query`, and have the highest search count (most popular)	Fix typos/Spell-correct the user's partialQuery before showing the typeahead results	While showing the suggestions, the part that has already been typed should be highlighted (frontend)
With every letter that the user types, the typeahead suggestions should change *we will update the typeahead with every letter - send a new API request with every letter typed*	Even when the user has typed `< 3` characters, we should suggest the	after giving suggestions, if user press tab button, it

	"trending" global searches / previous recent searches by the user	should write copy the query in the text box (frontend)

We will start showing typeahead suggestions only
● after the user has typed at least `3 letters`
  ○ otherwise, we will probably show irrelevant results
● if the user's query is at most `50 letters`
  ○ most likely, they've slept on the keyboard
		user age based personalization
		
		query: ***why is s***
		5 years => ***why is s****ky blue*
		30 years => ***why is s****cotch smooth*

		Grammar-correct the user's partialQuery before showing the typeahead results
		provide suggestions as per the *law of the land* *(what??)*

What to avoid

1. anything that doesn't effect our backend infra / system design
2. frontend features
   a. don't effect our backend infra / system design
   b. purely cosmetic
3. common features that are not the core of the system that we're trying to build
   a. design Uber
      i. we require **authentication**
      ii. is authentication what the interviewer is asking you to design
      no - it's just a dependency
      iii. we require **payments**
      iv. when the interviewer asks us to design Uber, they're not asking us to design payments
      v. we will assume that auth & payments already exist
4. any suggested feature should come in the form of a complete sentence (from the user's perspective)

Hidden features

How to find hidden features?

For every feature that you discuss, think of the following

API (internal / user-facing)

```
typeahead(partial_query) => suggestions_list
```
user-facing

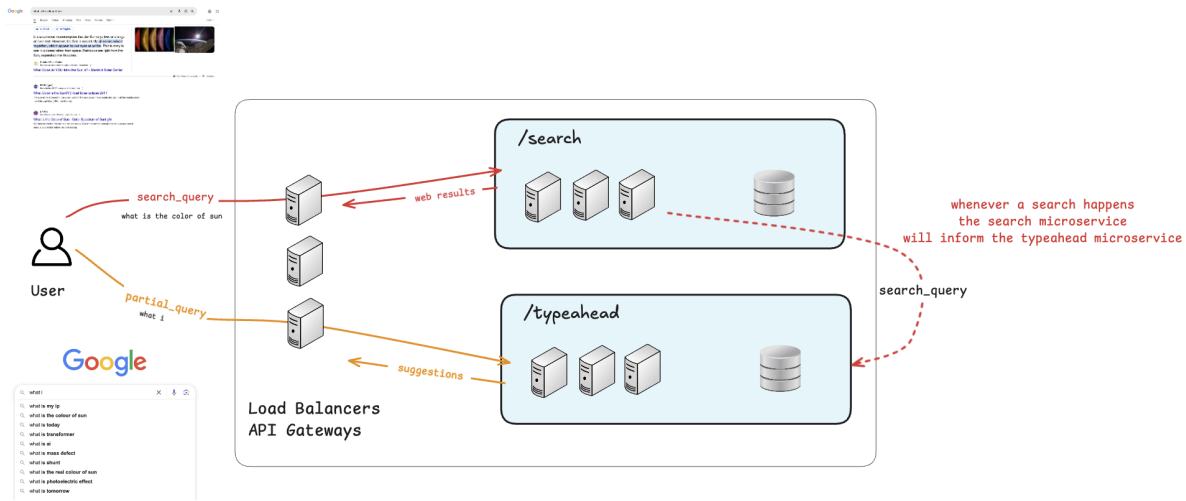
`log_search(search_query) => void`
internal

~~GET /typeahead?partialQuery={}~~
- ~~why REST why not SOAP / RPC / ...~~
- ~~why GET, why not POST~~
- ~~naming convention~~

API: input & output (interface)
Protocol: how the communication happens (REST / ...)

Data Source & Flow

Where does the list of suggestions come from? How is the “search query” data getting populated?



Users are constantly searching on Google.
Every time a user makes a search (search_query)

- this search_query goes to the /search microservice
- search microservice will return the relevant web-results
- (async) the search microservice will inform the /typeahead microservice that this search has been made (pass the search_query)
- the /typeahead service can store this search_query to populate its suggestions list

Non-Functional Requirements (NFR)

(design goals)

PACELC

Consistency vs Availability

Do NOT jump to an answer

Q: What is the data ?

Past search queries and their corresponding counts.

Search Query	Count
why is the sky blue?	49,999
why is water wet?	50,000
what is the color of the ocean	5,000

Q: What does eventual consistency mean in this context ?

When is the data changing?
Everytime someone makes a search

- the count of some entry changes
- the top-5 relevant results for some typeahead partial_query changes

It is possible for the search-count to reflect in a delayed manner in the top-5 results.
For some time, it is possible that the true count of Query1 > Query2, but we still show Query2 above Query1 in the typeahead suggestions.

Q: Can we afford stale reads in this situation?

Yes, that's okay

1. because the user doesn't really know (or care about) the true search counts
2. we don't necessarily need to maintain a strict order in the typeahead suggestions
    a. it is okay for a few good suggestions to not be shown, as long as the ones shown as also good enough

Q: What does data loss mean in this context ?

Data loss would happen if someone has searched for a query, but for whatever reason, we were not able to increment the count for that query (query didn't get logged).
The counts will be off by a small amount.

Search Query	True Count
why is the sky blue?	49,999
why is water wet?	50,000

what is the color of the ocean	5,000

Search Query	DB Count
why is the sky blue?	49,999
why is water wet?	49,998 (no issues)
what is the color of the ocean	5,000

Q: Can we afford data loss in this situation?

Yes, that's okay.
(same reasons as above)

Consistency vs Latency

What are our latency requirements?

Ultra-low latency — literally competing with the user's typing speed!
each typeahead query to take **< 10ms**
(this doesn't include the round trip - that depends on the user's distance from the servers - don't have much control over that - only solution for that is to have geolocated servers)

Others

- Security: do NOT do this unless you're a security expert
    - 99% of all security issues happen because non-security expert devs try to do security
- Observability
- Rate Limits
- Idempotency
- Delivery Guarantees

Scale Estimation

The scale will dictate our design choices

Goals

1. Amount of Data *(in bytes)*
    - Is sharding needed?

2. Amount of Load
    - Requests per second *(for each API)*
3. Peak Load (vs avg load)
4. Read-heavy / Write-heavy

All these things will help us guide our choice of database, cache, & sharding

Does accuracy matter?

No. Back-of-the-envelope estimates / approximations
being off by a factor of 2 or 3 is okay.
Actual value: 100
You suggest: 50 okay
You suggest: 200 okay
You suggest: 10 not okay (off by a factor of 10)

However, any estimates that you make should be justified. Don't pull out numbers from thin air. You will have to make some assumptions - each assumption should be clearly stated as such.

How to start

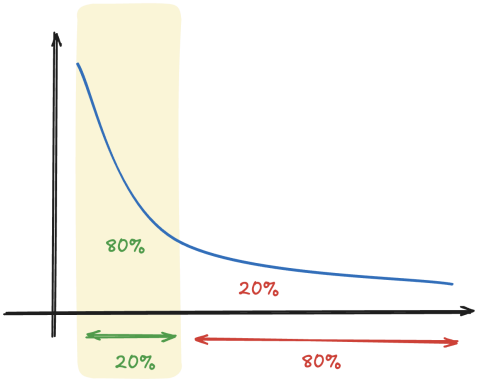
Start by estimating the number of Daily Active Users (DAU).

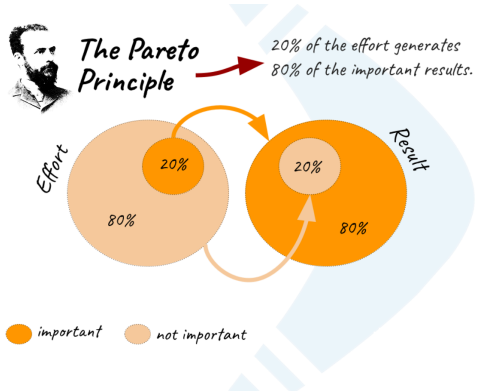
World Population: `~8 billion`
Number of Internet Users: `~5 billion`
India's Population: `~1.4 billion`
USA's Population: `~350 million`

Pareto Principle (the 80/20 rule)

Only 20% of your users will be active.
Can be extended to the 80-20-1 principle for social media

- 80% are just passively browsing
- 20% are interacting (comments, likes, shares)
- 1% are content creators (posts)





Fun stuff — Pareto principle & Zipf's Law: *The Zipf Mystery*

Estimates

assumption: Google has `5 billion` users.

Daily Active Users (DAU): `20%`
`= 20% of 5 billion users`
`= 1 billion users`

assumption: Number of searches per active-user per day: `20`

Total number of searches / day
`= (20 searches / user / day) * (1 billion users)`
`= 20 searches / day * 1 billion`
`= 20 billion searches / day`

Avg. searches / second
`= 20 billion searches / day`
`=` $20 * 10^9$ `searches /` $(10^5$ `seconds)` (1 day = 86400 seconds ~ 10^5 seconds)
`= 200,000 searches / second`

Note: searches are going to search API, not the typeahead API

assumption: Number of typeahead queries per search query:
assumption: avg size of a search query is `10 letters`
we will start showing suggestions after the first 3 letters
avg. number of typeaheads per search would be `(10 - 3) = 7 ~ 10`

`typeahead(...)` API: requests / second

Everytime a user is searching for something, they will first type the query, and then they will search.

search_query: "why is the sky blue"

1. typeahead("why")
2. typeahead("why i")
3. typeahead("why is")
4. typeahead("why is t")
5. typeahead("why is th")
6. typeahead("why is the")
7. typeahead("why is the s")
8. ...
9. typeahead("why is the sky blue")

`= (200,000 searches / second) * (10 typeaheads / search)`

`= 2 million typeaheads / second`

`log_search(...)` API: requests / second

Every time a search is made, the /search microservice pings the /typeahead microservice

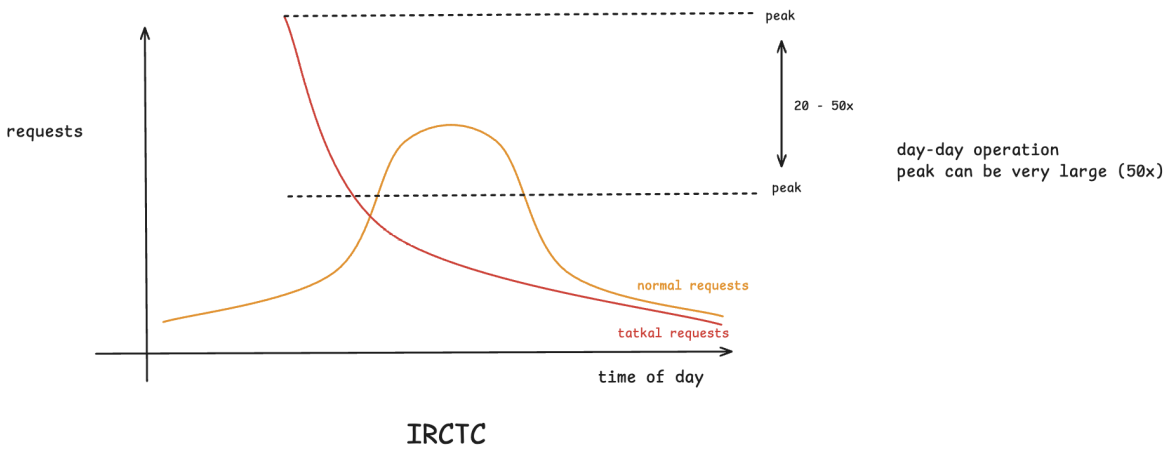
`= 200,000 requests / second`

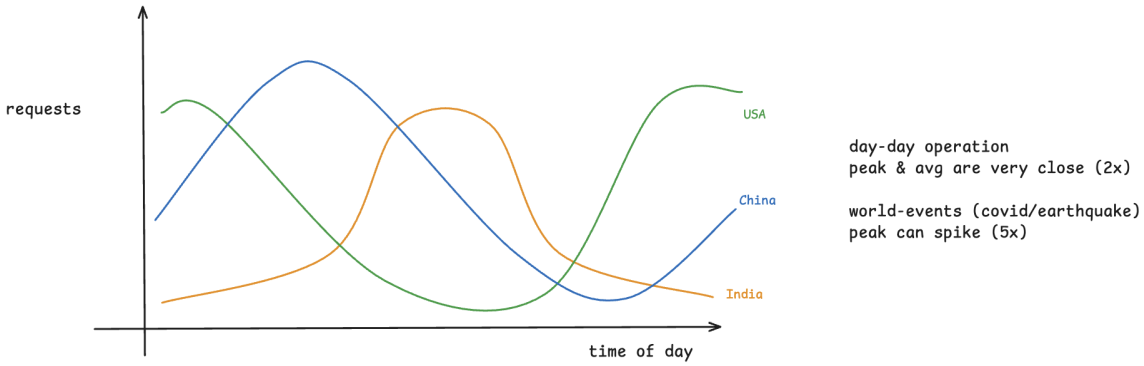
Peak Load

assumption: peak load is `5x` the average load

`= 5 * (2 million typeaheads / second)`

`= 10 million typeaheads / second`





Google Typeahead

Amount of Data

What's the data?
Past search queries & their counts

Search Query	True Count
why is the sky blue?	49,999
why is water wet?	50,000
what is the color of the ocean	5,000

Avg. size of search query: `10 letters` (made this assumption earlier)
How many entries in this table?

Should we say that for each search that happens, we will have an entry in this table?

No. Because for new searches, we're adding entries
But for queries that have already been searched earlier, we're just updating the counts

The number of entries in the table will depend on the number of "unique" (never seen before in the history of Google) search queries that happen

~~**assumption:** % of search queries are new (never seen before) = 0.1%~~
assumption: % of search queries are new (never seen before) = 10%

number of new entries / day
= 10 % of (20 billion searches / day)
= 2 billion entries per day

amount of data / day

```
= (20 bytes / entry) * (2 billion entries / day)
= 40 GB / day
```

total amount of data (over 20 years)

```
= (40 GB / day) * (20 years)
= (40 GB / day) * (20 * 400 days)      (365 ~= 400)
= 32 * 10^4 GB
= 320 TB
```

Can this amount of data fit on a single server?

Probably yes, with modern storage.
But, will a single server be able to handle 10 million requests / second?
Absolutely no!

We need sharding!

Read Heavy / Write Heavy?

`typeahead` => read query (show suggestions) => `10 million requests / second peak load`
`log_search` => write query (incr count) => `1 million requests / second peak load`

reads are 10x more than writes

We will call this **read heavy**, because even though the system has to handle lots of writes too, it is dominated by reads.

Whenever you've read heavy systems & the writes are also significant

- you can absorb your reads in a cache
- optimize your DB for writes

No database system is optimized for both reads & writes (that's impossible)

System Design

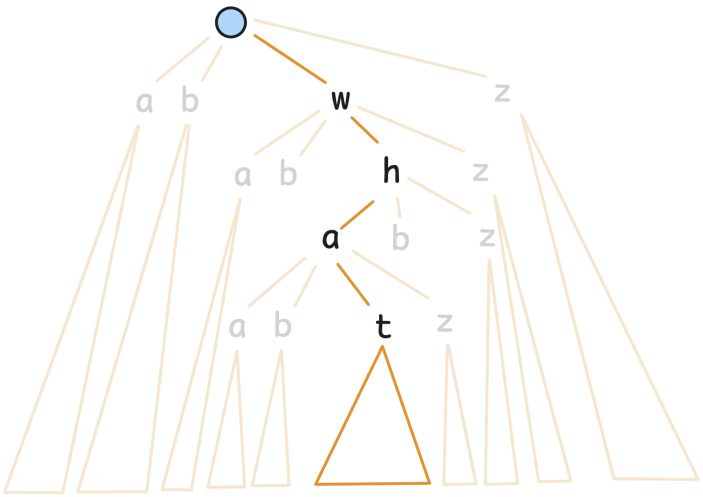
Typical Components

- Microservices

- Load Balancer
- Database choice
    - SQL
    - NoSQL
        - justify
- Cache choice
    - Need for caching
    - Type of cache
        - Local vs Global
        - (if global) Single vs Distributed
    - Invalidation
    - Eviction
- Communication
    - user - backend
    - backend - backend
- CDN
- Observability

~~Approach 1 - Tries~~

If the user types the partial query *"What"*, then we must show the top 5 search queries that are a strict prefix match.



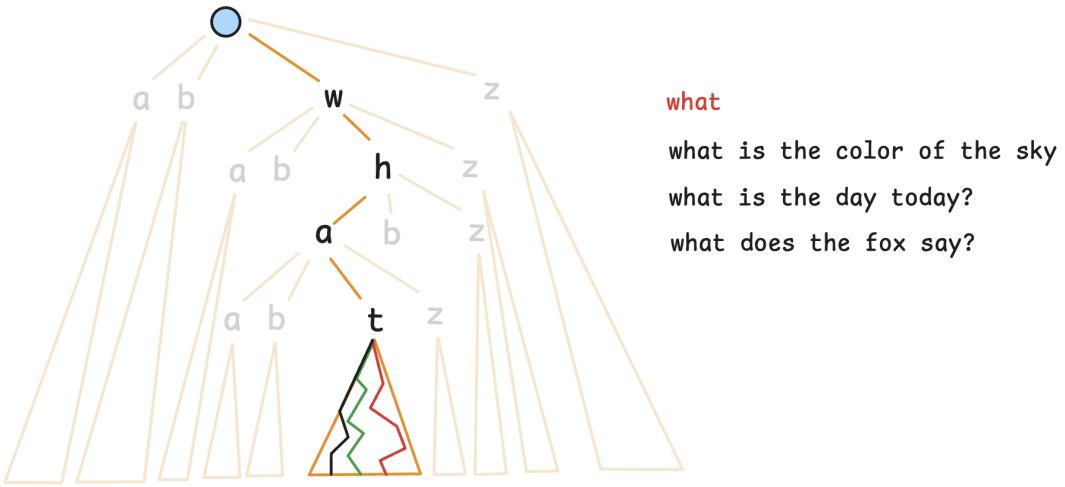
Each trie node will store

1. children (all the subsequent letters)
2. isTerminal
3. count (*only if isTerminal* — the number of times this search query has been searched)

Queries

`typeahead(partialQuery)`

1. go to the node that matches the partialQuery
    a. start from root
    b. for each letter of the partialQuery, we will go to the appropriate child
2. Once we've reached the matching node, all the possible suggestions are within the subtree of this node
3. We will have to go through the entire subtree (all possible suggestions that match the prefix), and find the top 5



Going through the entire subtree of a node will be highly time consuming
There's potentially billions of entries in that subtree!
Can improve this by "data-augmentation"
At every node, pre-compute & store the top 5 results.
In this case, the updates will become slow

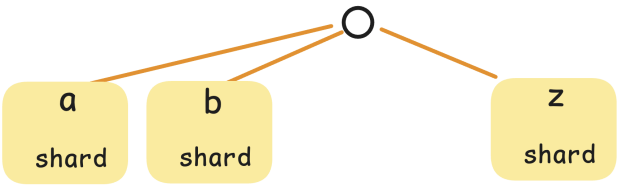
`updateCount(searchQuery)`

1. traverse to the node that matches this searchQuery
2. increment the count

Sharding

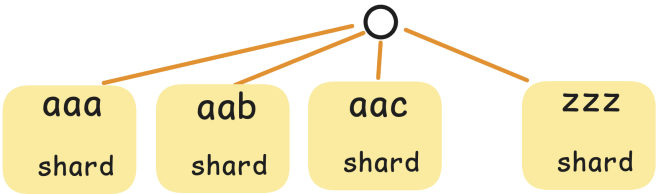
Q: What will be the sharding key?

1. First letter: every query that starts with the letter a will go to the same shard
   this doesn't mean that the starting letter a must get a dedicated shard - a single shard might contain multiple starting letters. We're just saying that all queries that start with the letter a must definitely go in the same shard.

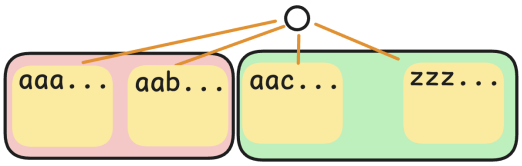


a.
b. Low cardinality *(only 26 possible children)*
c. Uneven data distribution
number of queries starting with `a` *will be much higher than the number of queries starting with* `z`

2. First 3 letters: every query that starts with the letters `abc` will go to the same shard
this doesn't mean that the starting letters `abc` *must get a dedicated shard - a single shard might contain multiple starting letters. We're just saying that all queries that start with the letters* `abc` *must definitely go in the same shard.*



a.
b. High cardinality: 26^3 `~= 17000`
c. Uneven data distribution
number of queries starting with `why` *will be much higher than the number of queries starting with* `zxz`
   i. to fix the load distribution, we will have to manually create groups of starting 3 letters in order to club the less frequent queries together
   ii. *number of queries starting with* `why` *will be much higher => dedicated shard*
   iii. *number of queries starting with* `zxz` *will be lower in count, so we will group them with other rare starting letters* `zyx + zxy + zxz + ..`



Q: Which existing Databases support this?

There's no popular database that was built for storing tries!

Tries exist as an 3rd party extension in some popular databases. However, no database was built specifically to store tries.

So you must build your own DB.

Approach 2 - Hashmap / Key-Value

Search Frequency DB

for each actual `searchQuery` it will store the count

Search Query	Count
what is the color of the sky	5000
what is the day today	1000
what is 2 + 2	10000
what does the fox say	2000
what does a fox eat?	1900
how to kill someone	2000
how to cook eggs	1500
how to sing	500

Top Suggestions DB
(cache)

for each possible `prefix` *(for which our system will show suggestions)*, it will store the top k suggestions

Prefix (partial query)	**Top k** `(=3)` **suggestions**

wha	`[`
	`what is 2+2 => 10000`
	`what is the color of the sky => 5000`
	`what does the fox say => 2000`
	`]`

what	`[`
	`what is 2+2 => 10000`
	`what is the color of the sky => 5000`
	`what does the fox say => 2000`
	`]`

...	

what i	`[`
	`what is 2+2 => 10000`
	`what is the color of the sky => 5000`
	`what is the day today => 1000`
	`]`

what d	`[`
	`what does the fox say => 2000`
	`what does a fox eat? => 1500`
	`]`

how	`[`
	`how to kill someone => 2000`
	`how to cook eggs => 1500`
	`how to sing => 500`
	`]`

...	

how to k	`[`
	`how to kill someone => 2000`
	`]`

Queries

`typeahead(partial_query)`

If someone types `"what i"` then we can just go to the suggestions db, and look up that partialQuery.

This will be very fast

1. hashmaps have `O(1)` lookup
2. we don't need any computation / processing - we just have to get the value for the given key from the key-value DB

We needed **low latency** (< 10ms), and, **high read throughput** (10 million reads / second at peak)

- Redis lookup has latency of <= 1ms
- A single redis server can easily handle 100,000+ reads/writes per second!
  - just need ~100 servers
    (google has over 10 million servers - as of 2020)

`log_search(search_query)`

Whenever someone searches for a query `"what does the fox say"`

1. I need to update the count in the search frequency database
   - super simple - just call `redis.inc(searchQuery)`
2. I need to update the cache (top suggestions database)
   - what all entries do I need to update?
     i. **If I'm updating the count for `what does the fox say` then is it possible for the suggestions of the prefix `"how"` to get changed?**
     No.
     ii. **What prefixes will get effected?**
     only the prefixes of the search query
     `wha`
     `what`
     `what d`
     `...`
     `what does the fox`
     `what does the fox s`
     `what does the fox sa`
     `what does the fox say`
     iii. **How many such prefixes (on average) need to be updated?**
     the average query is 10 letters. So ~10 prefixes on average need to

be updated.
but this now means, that for each update, I must do `1 + 10` writes
total number of writes in redis
`= (1 million log_search / sec) * 11 writes / log_search`
`= 11 million writes / second`
this makes my system both read & write heavy!
no db in the world that is optimized for both reads & writes

Search Frequency DB

Search Query	Count
what is the color of the sky	5000
what is the day today	1000
what is 2 + 2	10000
what does the fox say	~~**2000**~~ **2001**
what does a fox eat?	1900
how to kill someone	2000
how to cook eggs	1500
how to sing	500

Top Suggestions DB
(cache)

Prefix (partial query)	Top 3 suggestions
wha	[what is 2+2 => 10000 what is the color of the sky => 5000 what does the fox say => ~~2000~~ 2001]
what	[what is 2+2 => 10000 what is the color of the sky => 5000 ~~what does Shubadeep say => 2000~~ what does the fox say => 2001]
...	
what i	[what is 2+2 => 10000 what is the color of the sky => 5000 what is the day today => 1000]
what d	[what does the fox say => ~~2000~~ **2001** what does a fox eat? => 1500]
what do	[what does the fox say => ~~2000~~ **2001** what does a fox eat? => 1500]
how	[how to kill someone => 2000 how to cook eggs => 1500 how to sing => 500]
...	
how to k	[how to kill someone => 2000

	`]`

Sharding

Sharding is automatic - based on the `hash(key)`

Q: Are there databases that have 1st class support for hashmaps?

Yes. Not just 1 - there's an entire category!

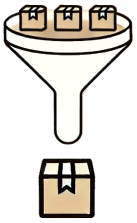
Q: Which existing Databases support this?

Key-Value databases are basically just hashmaps that are distributed across servers.
Redis / Memcached / DynamoDB / ...

Optimizing Writes

We can reduce the number of writes significantly, because, we can afford eventual consistency & data loss (non-functional requirements)

Batching / Batch Processing



Instead of doing each task 1 by 1, you wait for a lot of tasks to pile up. And then you do all of them in 1 go as a batch.

Whenever we get a `log_search(search_query)` request, instead of updating the suggestions db immediately, we wait for the count to increase by a fixed amount (say 1000)

Earlier

```
fn log_search(search_query):       // 1 million qps
    frequency_db.inc(search_query) // 1 million qps
    update_prefixes(search_query)

fn update_prefixes(search_query):
    for i = 3 ... len(search_query) - 1
        prefix = search_query[:i]
        ... logic to update the suggestions list  // 10 million qps
            in the suggestions db
```

Batching

```
fn log_search(search_query):       // 1 million qps
    updated_count = frequency_db.inc(search_query) // 1 million qps
    if updated_count % 1000 == 0:
          // update the prefixes only if the count
          // has changed by the batch size
         update_prefixes(search_query)

fn update_prefixes(search_query):
    ...                                // 10 million / 1000 qps
```

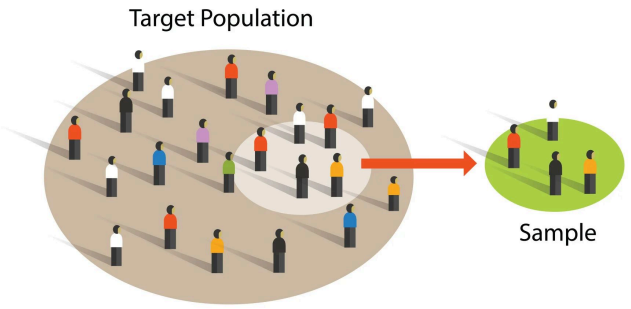
Effectively, we will only increase the counts in multiples of 1000.

Thanks to batching, instead of having `1 million + 10 million` writes/second, we now come back to just `1 million + (10 million / 1000)` = `1,010,000` writes / second

Note: we cannot make the number of writes less than 1 million per second, because that's the number of times we've the update at least the counts in the frequency database.

Note: batching doesn't cause data loss (freq data is always up-to-date) — it just causes delays (stale reads) in the suggestions updates.

Sampling



Exit polls during elections

1. country-wide election: 10 crore voters
2. the actual counting will take several days
3. News channels want to predict the winner with high accuracy before the actual results are announced
4. The news channels will ask various people "who did you vote for" and they will create their own voting list & winners
5. Can the news channels talk to all the 10 crore voters?
   No.
6. They stand in front of a small fraction of the polling booths
   As the people exit the booths, they will ask a few random people "who did you vote for"
7. They will only use this small subset/sample of data to "estimate" the winner
8. if the news channel is unbiased their exit poss results will almost perfectly match the actual election results

If you draw an unbiased sample from a population, then any trends that hold within the population will also hold within the sample.

Typically a good way of getting unbiased samples is to just sample uniformly at random.

Note: Sampling can cause data loss of individual data points. But it won't lose the overall trends. (use it only when data loss is okay)

Earlier

search service

```
fn search(query):
    ...
    make an API call to typeahead service's log_search endpoint
```

typeahead service

```
fn log_search(search_query):
    frequency_db.inc(search_query)
    update_prefixes(search_query)
```

```
fn update_prefixes(search_query):
    for i = 3 ... len(search_query) - 1
        prefix = search_query[:i]
        ... logic to update the suggestions list
            in the suggestions db
```

Sampling

search service

```
fn search(query):
    ...
    if rand() < 0.001:
        // with a probability of 1/1000, make the following call
        make an API call to typeahead service's log_search endpoint
```

typeahead service

```
    ...
```

For 99.9% of the searches, we're not even updating the counts (just ignore them)
For a random 0.1% of the searches, we're calling the log_search and we will update both the counts & the suggestions.

Thanks to sampling, instead of having `1 million + 10 million` writes/second, we now come back to just `(1 million + 10 million) / 1000` = **`10,000` writes / second**

Q: Won't the counts be inaccurate?

Yes. Effectively, each count is being divided by 1000 (but not exactly - it's random)

Q: Won't the suggestions be not exactly in order?

Yes.

But none of that matters. Because still, the suggestions shown to the users will continue to be highly relevant!

Q: If we use sampling, then won't infrequent queries have very low counts / might completely be missed out from the database?

Yes, that can happen! That's a feature, not a bug!
A rare query will never be a part of typeahead suggestions anyway (because it is not popular)

Effectively, sampling has automatically reduced your data massively by removing the majority (99.9%) of the bad queries 🙂

Future Scope

Recency Factor

Problem Statement

Suppose there's a big event that happens

- COVID lockdown gets declared
- Election results are out
- Some high profile person got arrested
- Federer won the Wimbledon
- ...

This is a trending event.

Even though `"who has control over nukes?"` might have a higher overall count, but since the query `"who won the Wimbledon?"` is trending *(recent + popular)*, we should rank that higher when someone types `"who"`

Goal

give more weightage to the recent counts

Solution

Simply decay the historical counts after every fixed period of time.
After each day, decrease the total count for each query by 10%

```
"who has control over nukes?"
[1000, 1000, 1000, 1000, ...]   => total is 100,000
day 1: 1000
day 2: 90% of 1000 + 1000 => 1900
day 3: 90% of 1900 + 1000 => 2710
day 4: 90% of 2710 + 1000 => 3439
...
day 100:  => 1000 + 1000 * 0.9 + 1000 * 0.9^2 + ...
          => 1000 * (1 + 0.9 + 0.9^2 + 0.9^3 + ...)
          => 1000 * 1 / (1 - 0.9)
          => 10,000

"who won the wimbledon"
[0, 0, 0, ... , 5000, 5000, 5000] => total is just 15000
day 997: 0
day 998: 5000
```

```
day 999: 5000 * 0.9 + 5000 => 9,500
day 1000: 9500 * 0.9 + 5000 => 13,550
```

If the count after decay reduces below a threshold (say 0) then we can remove that entry.

Geolocation based personalization

Given the user's request, find out their location based on ip address.
Build a separate database for each location => shard the DB by country_id
Now an Indian user's request will only go to the Indian shard.

User based personalization

Do this purely on the client side.
The browser stores the user's search/browsing history.

1. browser will create typeahead suggestions from the user's local data
2. browser will initiate a backend request to get the global typeahead suggestions
3. browser will merge these two lists

Handling Typos

You can simply have a dictionary and you can calculate the “edit-distance” b/w the user’s words and the words in the dictionary.
[Peter Norvig — How to Write a Spelling Corrector]

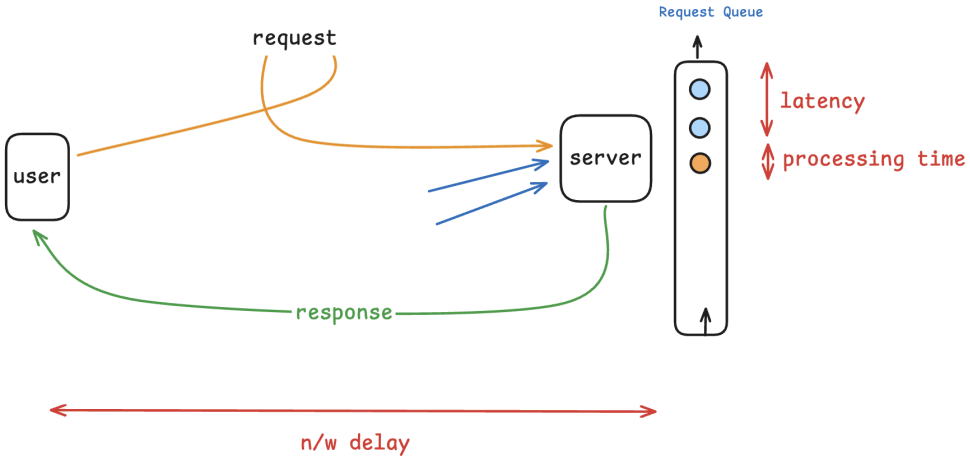
QnA

Round Trip Time / Latency / Throughput

Latency: the amount of time a requests spends waiting for processing
Processing time: the amount of time it takes for the server to process the request and generate a response
Round Trip Time: the total amount of time b/w the request & response

```
round trip time =  n/w delay + latency + processing time
```

Throughput: the number of requests the server handles per unit time *(per second)*



Usually, when people say latency, they mean the round trip time

A system can have any combination

- **High throughput, High Latency**
  Scaler code judge: each request takes ~5 seconds to process (because we've to run the testcases, and that is time consuming) => 5 seconds of latency **(high latency)**
  We can still handle 200 requests / second (by processing requests in parallel across multiple servers) => **high throughput**
- **High throughput, Low Latency**
  Scaler contest leaderboard => each request is just a redis lookup => **low latency**
  we can handle 100,000 requests / second => **high throughput**
- **Low throughput, High Latency**
  video processing requires a lot of time => **high latency**
  if you don't have a lot of servers to distribute the requests, the **throughput will be low**
- **Low throughput, Low Latency**
  ...

Writes are harder than reads
If the write volume is high => the database must be optimized for writes.
It is easier to absorb high reads => because you can just cache the data.

The only ways to improve write throughput

1. Batching => leads to stale reads
2. Sampling => can cause data loss
3. **Sharding** => increases latency for distributed transactions

Throughput per 1 server

SQL

1. simple reads => 10,000 per second
2. complex reads (involving joins / filters) => 100 .. 1,000 per second
3. simple writes => 100 ... 1000 per second
4. complex writes => 10 ... 100 per second

Key-Value (in-memory)

1. simple reads => 100,000+ per second
2. complex reads => not supported
3. simple writes => 100,000+ per second
4. complex writes => not supported

Key-Value (+ disk persistence)

1. simple reads => 100,000+ per second
2. complex reads => not supported
3. simple writes => 1000+ per second
4. complex writes => not supported

Document / Column Family will be similar to key-value with persistence for writes.

Can immutable data always be considered consistent?

"At how many Km/hour is your door cooking the color chess?"
Answer: not defined *(chess is not a color; even if it were, you can't cook colors; even if you could, doors don't cook; even if they did, your won't measure the cooking speed in Km/hr)*
"What's the color of an electron?"
Answer: not defined *(color is a perception that happens when light of a certain wavelength falls on your retina; electrons can emit light of any wavelength depending on how they're de-excited)*
Similarly, **consistency isn't defined for data that doesn't change.**

Resources

1. **Spelling correction:** https://norvig.com/spell-correct.html
2. **Biases in media:** https://www.youtube.com/watch?v=_fHfgU8oMSo